class 5: Data Viz wih ggplot

Christopher Ko (200161129)

Table of contents

Background 1
Lets add some features (more layers) 4
Gene Expression Plot 6
Going Further 8
Limit to 1977 and 2007 11

Background

There are many *graphics systems* in R for making plots and figures.These include **"base R graphics"** like the `plot()` and add-on packages like **ggplot2**

Let's compare how we make a simple graph with these two systems:

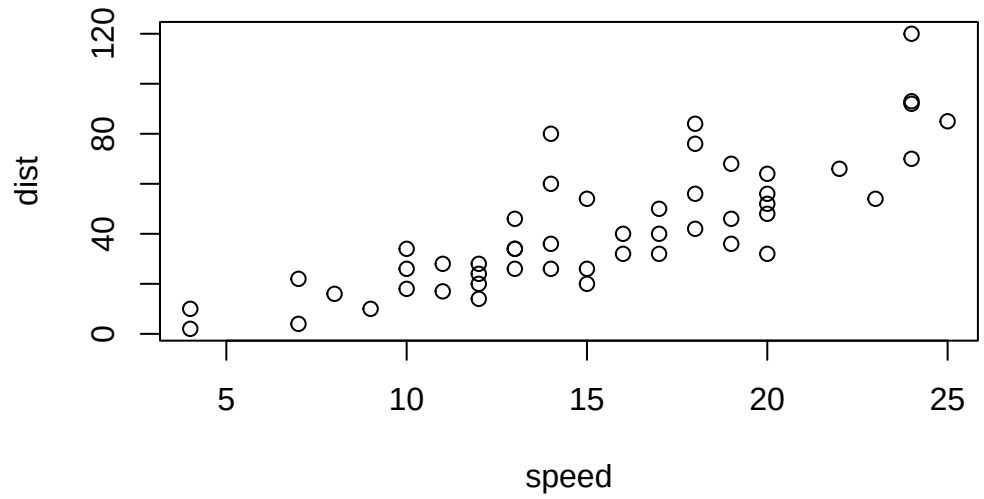
We can use the in-built `cars` dataset:

```
head(cars)
```

```
  speed dist
1     4    2
2     4   10
3     7    4
4     7   22
5     8   16
6     9   10
```

a base R plot:

```
plot(cars)
```



Now for **ggplot2**.

Before I can use ggplot2 I need to install it on my computer.To do this we can use the function `install.packages("ggplot2")`

> **N.B.** We never run an `install.packages()`in our quarto doc (we run t in our R console) as it would re-install the package every time I render my ny document

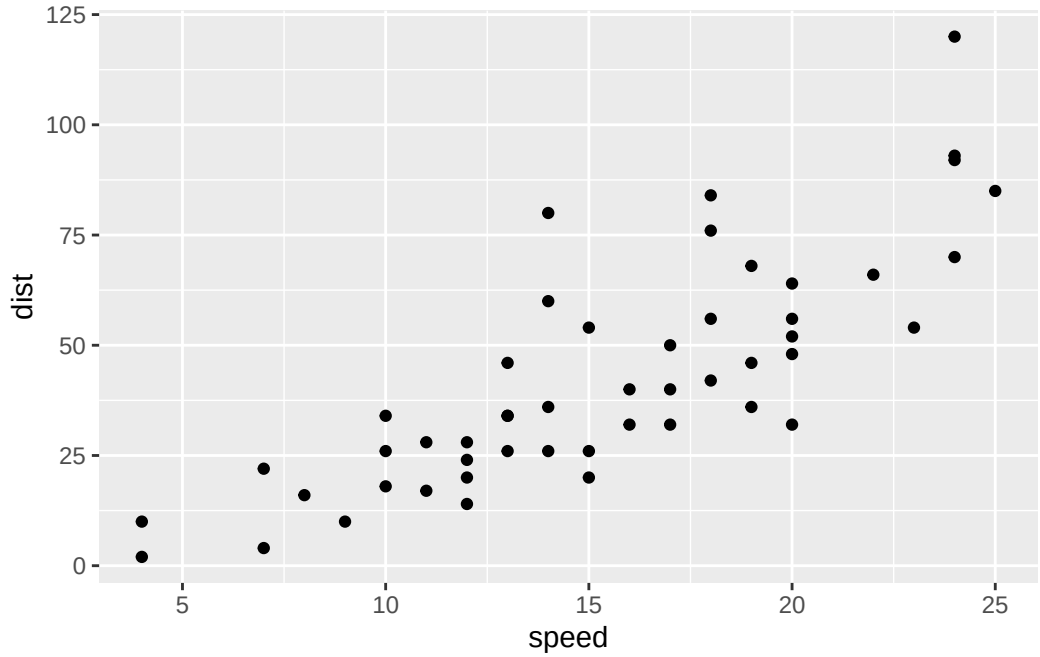
Once installed we still need to load up the package into our R session. We do this with a `library()` call

```
library(ggplot2)
ggplot(cars)
```

Every ggplot needs at least 3 inputs:

- the **data** (a data.frame of the stuff we want to plot)
- the **aesthetics** (how the data maps to the plot),
- the **geom** layer (how you want the plot drawn, points, lines, etc.)

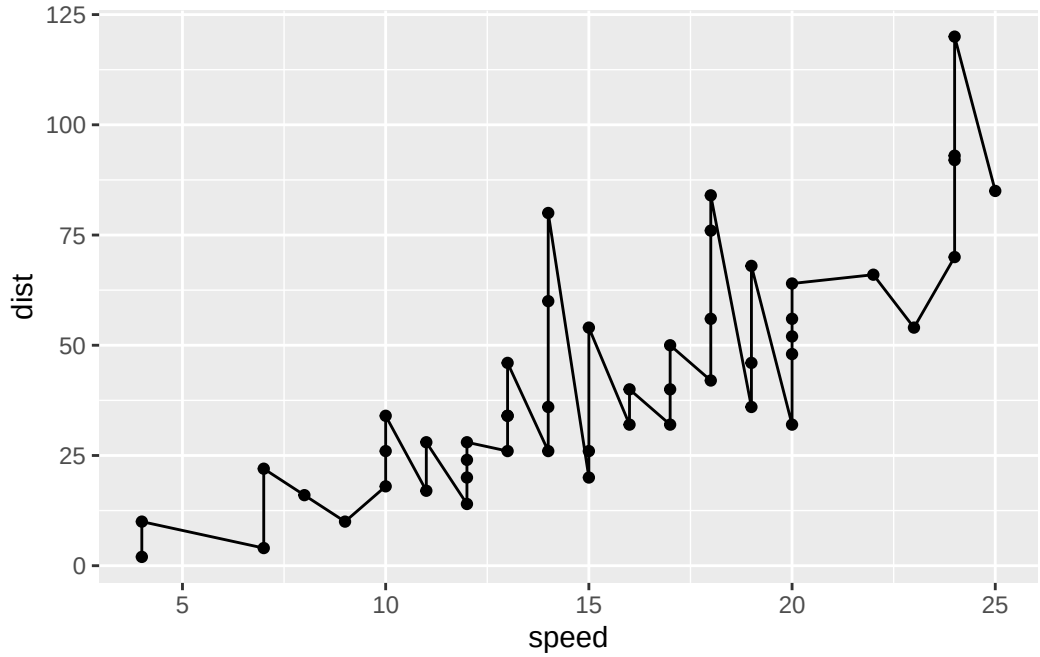
```
ggplot(cars) +
  aes(x=speed, y=dist) +
  geom_point()
```



Lets add some features (more layers)

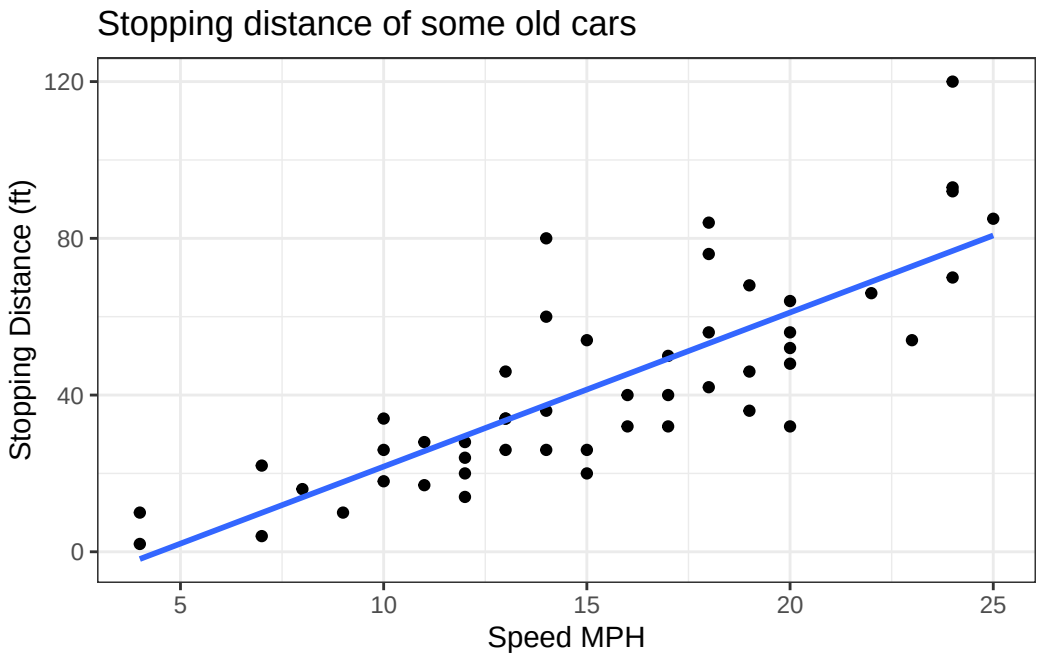
Let's add a trend line that shows the relationship between `speed` and `distance`:

```
ggplot(cars) +
  aes(x=speed, y=dist) +
  geom_point() +
  geom_line()
```




```
ggplot(cars) +
  aes(x=speed, y=dist) +
  geom_point() +
  geom_smooth(method="lm", se=FALSE) +
  theme_bw() +
  labs(x="Speed MPH",
       y="Stopping Distance (ft)",
       title= "Stopping distance of some old cars")
```

```
`geom_smooth()` using formula = 'y ~ x'
```



Gene Expression Plot

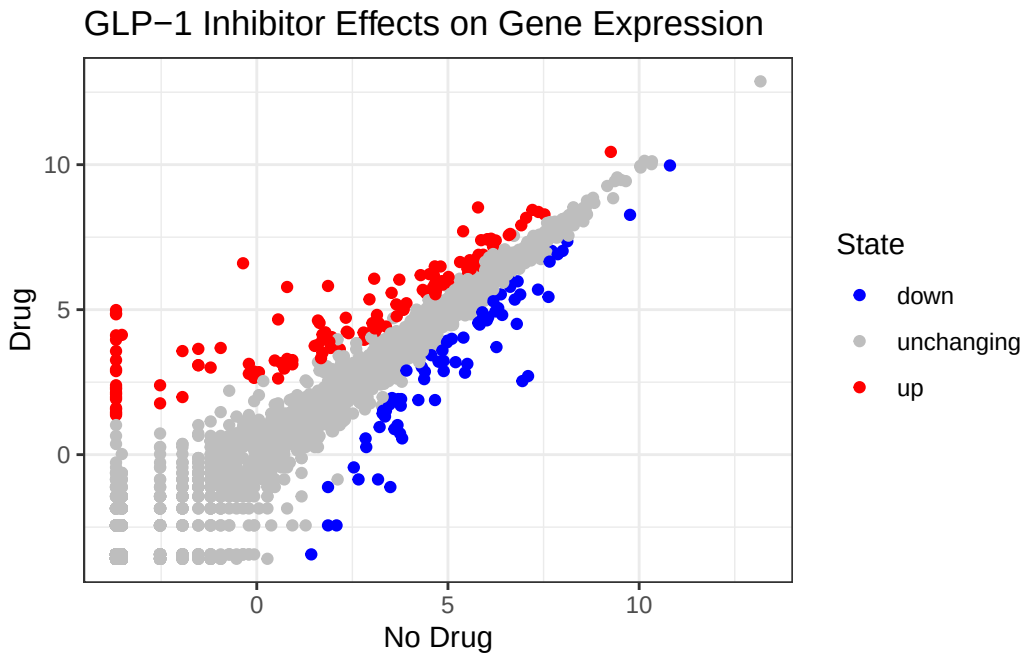
First we will read some expression data for the effects of a GLP-1 Drug

```
url <- "https://bioboot.github.io/bimm143_S20/class-material/up_down_expression.txt"
genes <- read.delim(url)
head(genes)
```

```
        Gene Condition1 Condition2      State
1      A4GNT -3.6808610 -3.4401355 unchanging
2       AAAS  4.5479580  4.3864126 unchanging
3      AASDH  3.7190695  3.4787276 unchanging
4       AATF  5.0784720  5.0151916 unchanging
5       AATK  0.4711421  0.5598642 unchanging
6 AB015752.4 -3.6808610 -3.5921390 unchanging
```

```
ggplot(genes) +
  aes(x=Condition1, y=Condition2, col=State) +
  geom_point() +
  scale_colour_manual( values=c("blue","gray","red") ) +
  theme_bw() +
```

```
  labs(x="No Drug",
       y="Drug",
       title= "GLP-1 Inhibitor Effects on Gene Expression")
```



Q. Use the nrow() function to find out how many genes are in this dataset. What is your answer?

```
nrow(genes)
```

```
[1] 5196
```

Q. Use the colnames() function and the ncol() function on the genes data frame to find out what the column names are (we will need these later) and how many columns there are. How many columns did you find?

```
colnames(genes)
```

```
[1] "Gene"       "Condition1" "Condition2" "State"
```

```
ncol(genes)
```

```
[1] 4
```

Q. Use the table() function on the State column of this data.frame to find out how many 'up' regulated genes there are. What is your answer?

```
table(genes$State)
```

```
      down unchanging         up
        72       4997        127
```

Q. Using your values above and 2 significant figures. What fraction of total genes is up-regulated in this dataset?

```
n.total <- nrow(genes)
n.up <- sum( genes$State == "up")

round(n.up/n.total*100, 2)
```

```
[1] 2.44
```

```
sum( genes$State=="up" )
```

```
[1] 127
```

Going Further

Here we read the famous *gapmider* dataset:

```
# File location online
url <- "https://raw.githubusercontent.com/jennybc/gapminder/master/inst/extdata/gapminder.tsv

gapminder <- read.delim(url)
head(gapminder)
```

```
      country continent year lifeExp      pop gdpPercap
1 Afghanistan      Asia 1952  28.801  8425333  779.4453
2 Afghanistan      Asia 1957  30.332  9240934  820.8530
3 Afghanistan      Asia 1962  31.997 10267083  853.1007
4 Afghanistan      Asia 1967  34.020 11537966  836.1971
5 Afghanistan      Asia 1972  36.088 13079460  739.9811
6 Afghanistan      Asia 1977  38.438 14880372  786.1134
```

Q. How many entries are in this dataset?

```
nrow(gapminder)
```

```
[1] 1704
```

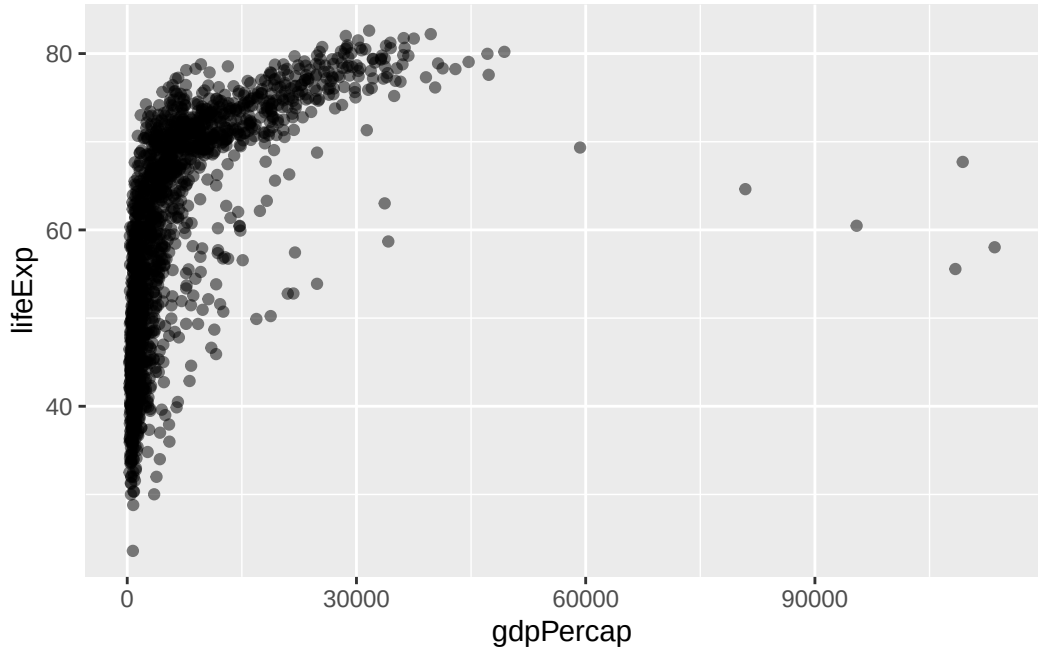
Q. How many different "country" values are in this dataset?

```
length( table(gapminder$country) )
```

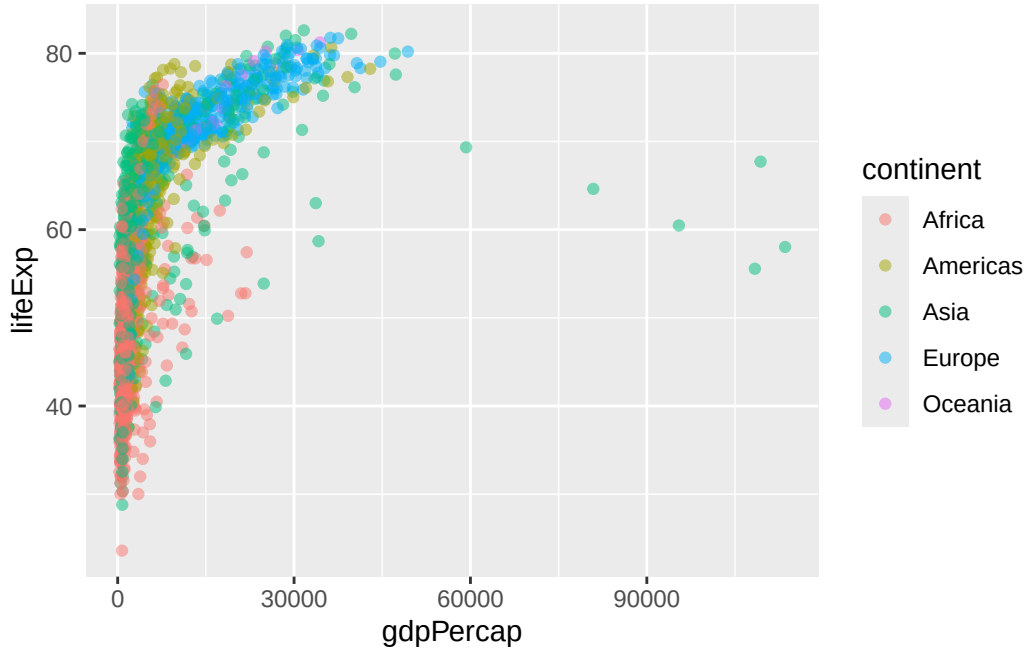
```
[1] 142
```

Let's make our first plot of `gdpPercap` vs `lifeExp`:

```
ggplot(gapminder) +
  aes(x = gdpPercap, y=lifeExp, ) +
  geom_point(alpha=0.5)
```

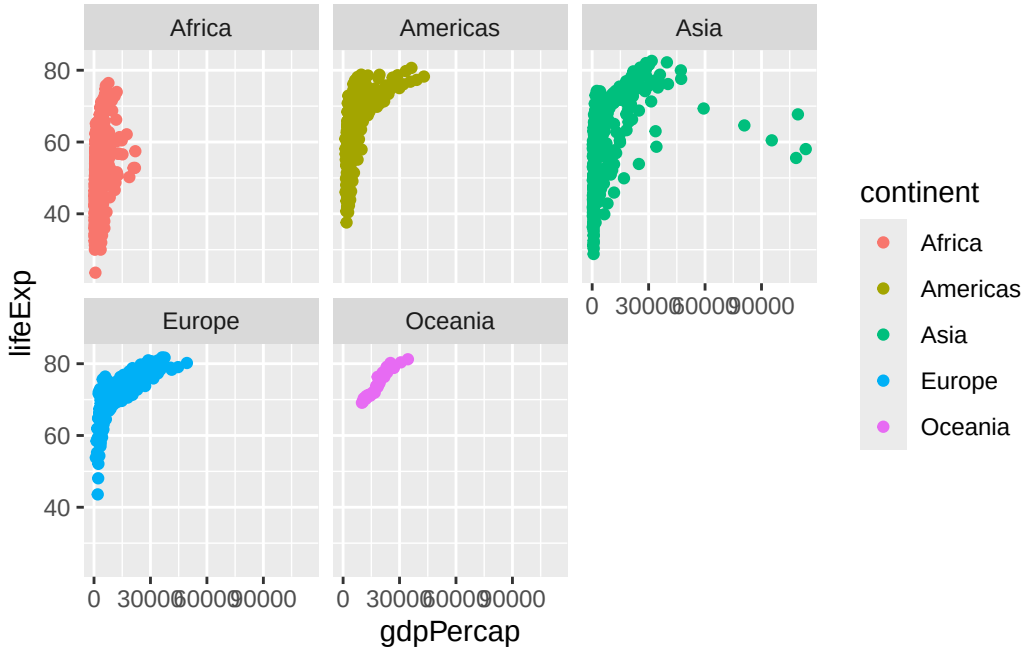



```
ggplot(gapminder) +
  aes(x = gdpPercap, y=lifeExp, col=continent) +
  geom_point(alpha=0.5)
```



One useful layer we can add to explore big datasets like this is called "facetting"

```
ggplot(gapminder) +
  aes(x = gdpPercap, y=lifeExp, col=continent) +
  geom_point() +
  facet_wrap(~continent)
```



Limit to 1977 and 2007

We can install and use the **dplyr** package to help us "filter" for the full dataset down to a smaller subset that only includes the years we want.

First we need to run `install.packages("dplyr")` and then load it with `library(dplyr)`.

```
library(dplyr)
```

```
Attaching package: 'dplyr'

The following objects are masked from 'package:stats':

    filter, lag
```

```
The following objects are masked from 'package:base':

    intersect, setdiff, setequal, union
```

```
filter(gapminder, year==2007, country=="Ireland")
```

```
  country continent year lifeExp     pop gdpPercap
1 Ireland    Europe 2007  78.885 4109086     40676
```

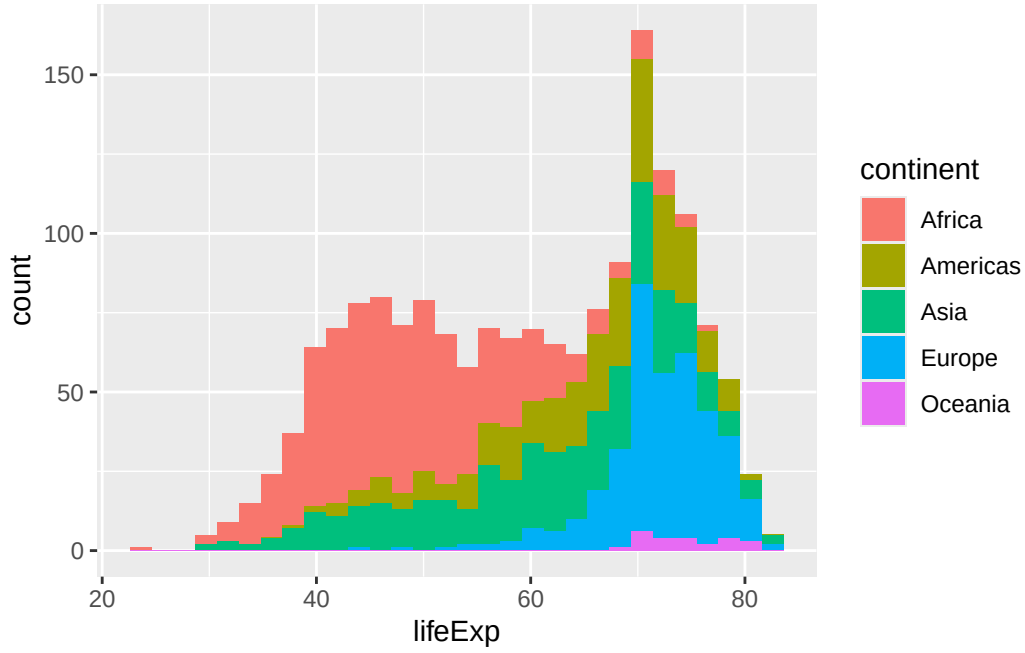
```
filter(gapminder, year==2007, country=="Ireland")
```

```
  country continent year lifeExp     pop gdpPercap
1 Ireland    Europe 2007  78.885 4109086     40676
```

Q. Make a histogram of lifeExp fill colorby continent

```
ggplot(gapminder) +
  aes(lifeExp, fill=continent) +
  geom_histogram()
```

```
`stat_bin()` using `bins = 30`. Pick better value `binwidth`.
```



```
ggplot(gapminder) +
  aes(lifeExp, fill=continent) +
  geom_boxplot()
```

